

Лекція 1.1

Тема: Заголовки Веб-сторінок. Стандартні атрибути елементів. Атрибути подій. Форматування тексту План лекції

1. Введення.
2. Анатомія Web-сторінки.
3. Правила синтаксису.
4. Основні елементи html.
5. Стандартні атрибути.
6. Атрибути подій.
7. Основні елементи для форматування тексту.
8. Табуляція, прогалини, переноси.
9. Гіперпосилання.

Зміст лекції

1. Введення.

Інтернет - це глобальна мережа, і тому вся інформація про мережу повинна бути представлена універсальною мовою, яку всі користувачі зрозуміли б. Мовою публікації, що використовується у всесвітній павутині, є HTML (мова Hyper Text Markup Language - мова розмітки гіпертексту). Технічну інформацію про цю мову (її специфікація) англійською мовою можна знайти на сайті - w3c.com

HTML - мова розмітки, надає розробникам такі можливості:

- Подавати інформацію у мережі у вигляді електронних документів, з інформаційним вмістом у вигляді форматowanego тексту, таблиць, списків, фотографій;
- Включати в документи звукові фрагменти, відеокліпи, електронні таблиці та інші програми та елементи мультимедіа;
- Здійснювати завантаження документів за допомогою активізації гіпертекстового посилання
- Розробляти форми для здійснення взаємодії з віддаленими службами (пошуковими роботами, on-line магазинами тощо)

2. Анатомія Web-сторінки

Нижче показана заготовка типового Web-документа. На цьому прикладі ми розглянемо структуру html-сторінок. Я назвав цей файл Strukt.htm.

Лістинг 1.1. Приклад (шаблон) Web-сторінки

```
<!DOCTYPE html>
<html>

<head>
  <Title> Структура Web-сторінки </title>
  <STYLE>
    H2 {
      font-family: Arbat;
    }

    CODE {
      font-family: Arial;
    }
  </style>
  <meta http-equiv="Content-Type" content="text/html; charset = UTF-8">
  <meta name="Author" content="Кожаєв Андрій">
```

[illegible]

існує велика різниця між стандартом офіційним і стандартом фактичним. html постійно розвивається, доповнюється новими елементами, і вивчати його треба не за офіційними першоджерел, а на практиці, звертаючись до останніх розробок провідних фірм і фахівців.

Щоб зрозуміти структуру Web-сторінки, необхідно розглянути всі елементи, що входять до наведеного вище лістинга. При розгляді елементів мови я буду приводити обидва тега: початковий і кінцевий. Наприклад: `<i> </i>`. Цим я хочу підкреслити, що в більшості випадків розробник повинен використовувати два тега для кожного елемента. Число випадків, коли допустимо тільки початковий тег (частина елементів не мають кінцевого взагалі), невелика, і вони спеціально обумовлюються. Для імен тегів можна використовувати як прописні, так і малі літери латинського алфавіту. Деякі користувачі записують початкові теги прописними буквами, а кінцеві теги - малими. Це допомагає розібратися в початковому тексті Web-сторінки.

`<html> </html>`

Позначення документа на мові html. Я вже згадував про те, що одним із принципів мови є багаторівневе вкладення елементів. Даний елемент є самим зовнішнім, так як між його початковим і кінцевим тегами повинна знаходитися вся Web-сторінка. В принципі, цей елемент можна розглядати як формальність. Він має атрибути *version, lang i dir*, якими в даному випадку рідко хто користується, і допускає вкладення елементів **head, body FRAMESET** і інших, що визначають загальну структуру Web-сторінки. Природно що кінцевим тегом `</html>` закінчуються всі подібні документи.

`<head> </head>`

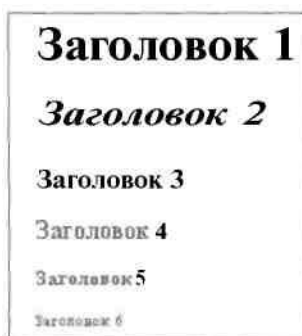
Область заголовка Web-сторінки. Іншими словами, її перша частина. Так само, як і попередній елемент, **head** служить тільки для формування загальної структура документа. Цей елемент може мати елемент **title** і допускає вкладення елементів **BASE, META, link, OBJECT, SCRIPT, STYLE**.

`<title> </title>`

Елемент для розміщення заголовка Web-сторінки. Рядок тексту, розташована всередині цього елемента, відображається не в документі, а в заголовку вікна браузера. Цей рядок часто використовується при організації пошуку в WWW. Тому автори, що створюють Web-сторінки для розміщення в Мережі, повинні подбати про те, щоб цей рядок, не був занадто довгим, досить точно відображав призначення документа.

`<style> </style>`

Опис стилю деяких елементів Web-сторінки. У файлі Strukt.htm призначені шрифти для елементів H2 і CODE. На рис. 1.1 видно, як зміниться вигляд заголовка другого рівня після такого перевизначення. Природно, що для кожного елемента існує стильове оформлення за замовчуванням, тому вживання елемента STYLE не обов'язково, але бажано.



Мал. 1.1. Заголовки, що створюються за допомогою елементів H1 ... H6. Шрифт другого заголовка перевизначений

`<META>`

Цей елемент містить службову інформацію, яка не відбивається при перегляді Web-сторінки. У середині нього немає тексту в звичайному розумінні, тому немає і кінцевого тега. Кожен елемент META містить два основних атрибуту, перший з яких визначає тип даних, а другий - зміст. Ось кілька прикладів meta-даних.

- Адреса електронної пошти: name = "Reply-to" content = "Ім'я@Адреса"
- Ім'я автора Web-сторінки: name = "Author" content = "name"
- Набір ключових слів для пошуку: name = "Keywords" content = " слово1, слово2, слово3 ..."
- Короткий опис змісту Web-сторінки: name = "Description" content = "Опис змісту сторінки сторінки"
- Опис типу і характеристик Web-сторінки: name = "Content-Type" content = "Опис сторінки"
- Програма, в якій була створена Web-сторінка: **name = "Generator"** content = "Назва html-редактора"

Тут і далі при описі елементів і їх атрибутів курсивом виділено фрагменти, які повинні бути заповнені користувачем на його розсуд. Атрибут **name** використовується додатком-клієнтом для отримання додаткової інформації про Web-сторінки і їх упорядкування. Цей атрибут часто замінюють атрибутом http-equiv. Він використовується сервером для створення додаткових полів при виконанні запиту.

Крім цього, елемент META може містити URL. Шаблон відповідного атрибута такий:

URL = "http://адреса"

<body> </body>

Цей елемент містить в собі гіпертекст, який визначає власне Web-сторінку. Це та довільна частина документа, яку розробляє автор сторінки і яка відображається браузером. Відповідно, кінцевий тег цього елемента треба шукати в кінці html-файлу. У середині елемента body можна використовувати всі елементи, призначені для дизайну Web-сторінки. У середині початкового тега елемента body можна розташувати ряд атрибутів, що забезпечують установки для всієї сторінки повністю. Розглянемо їх по порядку.

Для дизайну - атрибут, що визначає фон сторінки:

background = "Шлях до файлу фону"

Більш просте оформлення фону зводиться до завдання його кольору:

bgcolor = "#RRGGBB"

Колір фону задається трьома двозарядними шістнадцятирічними числами, які визначають інтенсивність червоного, зеленого і синього кольорів відповідно. Більш детально про завдання квітів буде розказано нижче.

Обидва наведених вище атрибута не є альтернативними і часто використовуються спільно: якщо з яких-небудь причин не може бути знайдений малюнок фону, використовується колір.

Оскільки фон сторінки може змінюватися, необхідно мати можливість підбирати відповідний колір тексту. Для цього є наступний атрибут

text = "#RRGGBB"

Для завдання кольору тексту гіперпосилань використовується наступний атрибут:

link = "#RRGGBB"

Точно так само можна задати колір для переглянутих гіперпосилань:

vlink = "#RRGGBB"

Можна також вказати зміну кольору для останньої вибраної користувачем гіперпосилання:

alink = "#RRGGBB"

Гіпертекст, розташований всередині елемента body, може мати довільну структуру. Її визначають, в першу чергу, призначення Web-сторінки і фантазія розробника.

<!-- Коментар --->

У будь-якій мові програмування є конструкції, що дозволяють створювати довільні ремарки. html в цьому сенсі - не виняток. Текст, введений всередині цього елемента, ігнорується браузером. Ці елементи можуть розташовуватися в будь-якому місці Web-сторінки. Без закриває кутової дужки тут, мабуть, не обійтися: коментар повинен бути

відділений від основного тексту. Ознакою коментаря служить знак оклику, а текст коментаря повинен обрамлятися подвійними дефісом. наприклад:

```
<!-- Початок виведення таблиці -->
```

```
<h1> </h1>
```

Елемент заголовка. Існує шість рівнів заголовків, які позначаються H1 ... H6. Рівень 1 найбільший, а рівень 6 забезпечує найменший заголовок. Мал. 1.1 дає уявлення про відносні розміри букв в заголовках. Для заголовків можна використовувати атрибут, що задає вирівнювання вліво, по центру або вправо:

```
align = "left" align = "center" align = "right"
```

```
<hr>
```

Горизонтальна лінія (horizontal rule). Елемент не має кінцевого тега, але допускає ряд атрибутів для вирівнювання вліво, по центру, вправо, по ширині:

```
align = "left"
```

```
align = "center"
```

```
align = "right"
```

```
align = "justify"
```

Можна задавати товщину лінії:

```
size = товщина в пікселях
```

Можна керувати довжиною лінії:

```
Width = довжина в пікселях
```

```
width = довжина у відсотках%
```

Можна вибрати колір:

```
color = "колір"
```

```
<a> </a>
```

html-документ може бути дуже великим, і в цьому випадку користувачеві повинна бути надана можливість швидкого переміщення до потрібного розділу документа. Для цього можна використовувати механізм гіперпосилань. Необхідний '!' також в потрібних місцях тексту розставити відповідні мітки. Докладний гіперпосилання обговорюються в розділі «Гіперпосилання» глави 3, а тут ми розглянемо тільки шаблон для створення міток:

```
<A name = "мітка"> Довільний текст </A>
```

В цьому випадку цьому рядку документа присвоюється ім'я, і, отже, іншій частині документа або навіть на іншому документі може бути створена гіперпосилання, яка веде в цю точку.

Наприклад, для переходу всередині документа можна використовувати наступну конструкцію:

```
<p> Перехід до <A href = "#мітка"> міки </a> </p>
```

Кілька подібних рядків можуть утворити своєрідний зміст Web-сторінки, який можна розмістити на початку і в кінці документа.

```
<base>
```

Елемент для завдання базового адреси (URL) для посилань. Це дозволяє опускати початкову частину адреси в посиланнях документа. Для використання цього елемента необхідно використовувати наступну конструкцію:

```
<base href = "http://комп'ютер/">
```

Фрагмент адреси **путь1** не є обов'язковим. При формуванні повної адреси він буде відкинутий. Так, якщо в тексті документа зустрінесться відносне посилання

```
<A href = "шлях2/ім'я_документа2.htm"> Відомий текст посилання </a>
```

то воно буде відповідати URL

```
http://комп'ютер/шлях2/ім'я\_документа.htm
```

У тому випадку, коли треба поставити базову адресу для локального диску (наприклад D:), повинна бути використана така конструкція:

```
<base href = "file://D:\Шлях\">
```

Тоді при вказівці відносної посилання можна буде задавати не тільки ім'я файлу, але і імена папок, в яких він знаходиться. Іншими словами, шлях до файлів може бути розбитий на дві частини: абсолютну і відносну. Це корисно в тому випадку, коли для файлів, зазначених в документі, є загальний початковий фрагмент шляху.

У вираженні абсолютної посилання можна також опустити вказівку на схему доступу (file://). У цьому випадку буде враховуватися тільки ліва частина абсолютного посилання до першого лівого символу "\", тобто ім'я локального диска.

2. Правила синтаксису

Тепер, коли ми знаємо, як виглядає код Web-сторінки, можна зробити деякі узагальнюючі висновки щодо синтаксису **html**. При використанні кожного елемента важливо знати, які елементи можуть розташовуватися всередині нього і всередині яких елементів може знаходитися він сам. Так, взаємне розташування елементів **html**, **head**, **title** і **body** має бути стандартним на будь-якій сторінці, правда, в тих випадках, коли не використовуються фрейми.

Існують групи елементів, які використовуються спільно. До них відносяться елементи для створення таблиць, списків, фреймів.

Велика кількість елементів, які використовуються для форматування тексту, допускають найрізноманітніші варіанти вкладення. І самі вони обов'язково повинні розташовуватися всередині певних елементів. Тут треба керуватися здоровим глуздом: кожен елемент виконує задану функцію і має певну область дії.

У наведеному нижче прикладі є два абзаци (перший в зеленій рамці) і таблиця:

```
<p style = "border: 3px solid green"> Текст абзацу 1 </p>
```

```
<table> ... </table>
```

```
<p> Текст абзацу 2 </p>
```

Таблиця в даному випадку - незалежний елемент. Її можна, наприклад, вирівнювати незалежно від решти тексту.

Можна використовувати інший код:

```
<p style = "border: 3px solid green"> Текст абзацу 1
```

```
<table>. .. </Table></p>
```

```
<p> Текст абзацу 2 </p>
```

Зник кінцевий тег першого абзацу йде після таблиці. Тепер таблиця є частиною першого абзацу, і зелена рамка буде охоплювати таблицю і текст.

І навпаки, елемент P може перебувати всередині таблиці: наприклад, один елемент клітинки TD може містити кілька абзаців P.

Порушення правил вкладення - одна з найбільш поширених помилок при створенні Web-сторінок. Щоб уникнути таких помилок, можна користуватися редакторами гіпертексту, які автоматично контролюють виконання правил синтаксису. Нижче наведена рядок, що містить типову помилку вкладення елементів:

```
<h1> Заголовок 1
```

```
<h2> Заголовок 2 </h1> Заголовок 3 </h2>
```

Треба зауважити, що браузері побудовані таким чином, що вони «намагаються» не реагувати на помилки розмітки гіпертексту. Якщо сторінка може бути відображена, то вона виводиться на екран без будь-яких попереджувальних повідомлень. Програма інтерпретує помилково розставлені теги певним чином і формує зображення, слідуючи логіці, закладеній в неї розробниками. При цьому вид сторінки може і не відповідати задумом автора. І тільки в разі дуже серйозних помилок або явних протиріч браузер виводить повідомлення з неможливості відобразити сторінку. Непрямим ознакою помилки розмітки може служити поява на сторінці фрагментів коду html. Користувачі багато працюють з Інтернетом, напевно стикалися з такою ситуацією.

Правила синтаксису поширюються і на використання стартового і кінцевого тегів, атрибутів і вмісту елемента. Не плутайте поняття «елемент» та «тег». Елемент – це контейнер, що містить атрибути всередині стартового тега та корисну інформацію між

стартовим і кінцевим тегами. Тег - це конструкція, укладена в кутові дужки і використовується для позначення області дії елемента.

Деякі елементи не мають кінцевого тега. Очевидно, що елементу BR, що означає кінець рядка, не потрібен кінцевий тег. Деякі елементи можуть використовуватися з кінцевим тегом або без нього. Найяскравішим прикладом служить елемент абзацу P. Він може мати кінцевий тег, але якщо цей тег не заданий, то ознакою закінчення дії елемента служить наступний елемент, який може логічно визначити кінець поточного абзацу: інший елемент P, елемент малюнка IMG, елемент списку UL, елемент таблиці table і т. д.

Таким чином, корисна інформація одного елемента повинна перебувати або між початковим і кінцевим тегами даного елемента, або між початковим тегом даного і початковим тегом наступного елемента. Будь довільний текст, введений на сторінку, сприймається оглядачем як підлягає висновку на екран і, отже, форматування відповідно до навколишніх цей текст елементами. При цьому не враховується розбиття тексту на рядки, отримане в текстовому редакторі. Теоретично, всю Web-сторінку можна вмістити в одній довгій рядку. Символи кінця рядка, введені, наприклад, в Блокноті, можуть допомогти читання коду html, але не відображаються браузером. Останній, при виведенні сторінки на екран, може обірвати рядок у відповідності з розстановкою елементів Hn, P або BR, а в інших випадках він форматує абзаци довільно, залежно від обсягу тексту, розміру шрифту і поточного розміру вікна. Тому Web-сторінки треба компонувати таким способом, щоб їх вигляд кардинально не змінювався для різних режимів дозволу монітора, розміру екрана, розміру вікна браузера, а також для повноекранного або віконного режимів.

Дуже важливим правилом, яке не має винятків, є розміщення атрибутів елемента всередині початкового тега.

3. Основні елементи html версії 4

<title> </title>

Елемент title визначає текст, який з'являється в заголовку вікна браузера під час перегляду сторінки. Цей текст не тільки служить підказкою, але може використовуватися і пошуковими машинами для аналізу сторінок.

Існує три способи для пошуку сторінок в Інтернеті на основі текстових даних: за ключовими словами елемента META, за текстом, розміщеному на сторінці, і по рядку заголовка всередині елемента title.

< style> </style> i <link>

Елемент STYLE теж повинен розташовуватися всередині елемента head. Якщо ви хочете розібратися, які нестандартні формати використовуються на сторінці, треба переглянути вміст цього елемента. У ньому будуть вказані необхідні формати. Якщо таких форматів немає, значить стилі сторінки записані в окремому файлі. Посилання на такий файл має перебувати в елементі link. Детальніше про стилях розповідається нижче в розділі «Таблиці стилів».

<meta>

Секція заголовка може містити кілька елементів META, кожен з яких відповідає за певний набір параметрів. Використання елементів META не є обов'язковим, але деякі настройки можуть бути дуже важливі. Так, наприклад, відомо, що браузер в деяких випадках здатний автоматично визначити вид кодування сторінки. Користувач, працюючи з браузером, може вибрати в меню певну кодування. Щоб виключити невизначеність при перегляді конкретної сторінки, на ній доцільно розмістити вказівку на кодову сторінку. Для документів в кодуванні Windows воно повинно бути таким:

<meta http-equiv = "Content-Type" content = "text / html; charset = windows-1251">

Інформація, зосереджена в елементах META, визначає загальні налаштування Web-сторінки і називається профілем. Профілі можна зберігати в окремих файлах і приєднувати до певної сторінки за допомогою спеціального атрибута елемента head:

<head profile = "W?!">

У секції head можуть розташовуватися елементи, які мають відношення до всієї сторінки цілком. Так, якщо для останньої створено звуковий супровід, то його параметри визначає елемент BGSOUND (див. Нижче розділ «Застарілі і нестандартні елементи >>»).

4. Стандартні атрибути

Існує ряд атрибутів, які можуть використовуватися в багатьох елементах. Частина цих атрибутів дуже важлива для конструювання Web-сторінок, а частина підходить тільки для вирішення певних завдань. -

Атрибут id виконує функції унікального імені елемента. Залежно від типу елемента, цей атрибут виконує різні функції (див. Розділ «Таблиці стилів» поточної глави і розділ «Елементи форм» глави 4). Атрибут classid задає програму або об'єкт, які можуть використовуватися в певних елементах.

Атрибут style може використовуватися з багатьма елементами. Він призначений для визначення формату конкретного елемента і може приймати самі різні значення. Детально він розглянутий нижче в розділі «Таблиці стилів».

Схожі функції виконує атрибут class. Його можна вказувати, якщо в секції head розташований елемент STYLE або використано посилання на каскадну таблицю стилів (див. Нижче розділ «Таблиці стилів»).

Атрибут align використовується для вирівнювання тексту, об'єктів або елементів цілком. Вирівнювання може виконуватися щодо меж вікна, рамки таблиці і т. Д. Кожен елемент дозволяє вказувати певні значення для цього атрибута. У загальному випадку значення можуть бути такі:

- left- вирівнювання по лівому краю;
- right - вирівнювання по правому краю;
- justify - вирівнювання по ширині (для тексту);
- center - вирівнювання по центру (по горизонталі);
- middle - вирівнювання по центру (по вертикалі);
- top - вирівнювання по верхній межі;
- bottom - вирівнювання по нижній межі.

Атрибут lang визначає, якою мовою набраний текст у поточному елементу: 1алд = ЧОД мови "Ось деякі коди:

- ar - арабська;
- de - німецький;
- el - грецький;
- en - англійська;
- en-us - американська версія англійської мови;
- es - іспанська;
- fr - французький;
- he - іврит;
- hi - хінді;
- it - італійський;
- ja - Японські;
- nl - голландський;
- pt - португальська;
- ru - російська;
- zh - китайський.

У деяких мовах застосовується зворотне звичного напрямку тексту: справа наліво. За допомогою атрибута DIR можна вказати необхідний напрям:

- dir = "LTR" - зліва направо;
- dir = "RTL" - справа наліво.

Атрибут `dir` теоретично може використовуватися в різних елементах, але не всі браузері забезпечують його роботу. Більш надійний спосіб - застосування спеціально передбаченого для подібних випадків елемента `BDO` (див. Нижче).

Атрибут `type` визначає тип документа, який вказується в посиланні. Тут використовуються так звані типи MIME (Multipurpose Internet Mail Extensions).

Спочатку вони призначалися для визначення формату відправлень електронної пошти, але зараз служать для вказівки форматів документів у складі Web-сторінок. Найбільш часто використовувані типи:

- `text / plain` - звичайний текст;
- `text / ess` - каскадний таблиця стилів;
- `text / html` - документ у форматі `html`;
- `application / postscript` - документ у форматі `PostScript`;
- `image / gif`, `image / j pg`, `image / png` - зображення у форматі `GIF`, `JPG` або `PNG`

відповідно; "

- `video / mpeg` - відеоролик;
- `application / Java` - аплет;
- `text / javascript` - програма (сценарій) на `JavaScript`;
- `text / vbscript` - програма (сценарій) на `VBScript`.

Для форм атрибут `type` має інший зміст: тип певного елемента форми (кнопка, поле введення і т. Д.).

Атрибут `charset` необхідний там, де треба вказати вид кодування, наприклад:
`charset = "windows-1251"`

Атрибут `longdesc` (long description) може бути корисний в тих випадках, коли для якогось елемента необхідно використовувати опис (коментар) великого обсягу. Тоді документ приєднується за допомогою посилання:

`longdesc = "URL"`

Атрибут `title`, навпаки, дозволяє створити коротку підказку. Вона з'являється на екрані, коли користувач має покажчик миші над елементом. Значення атрибута - довільна текстова рядок.

5. Атрибути подій

Для сторінки можуть бути визначені програми, які виконуються тільки в разі певних дій користувача. В цьому випадку моменти запуску програм треба «прив'язати» до певних подій. Наприклад, якщо треба змінити зовнішній вигляд елемента, коли користувач вкаже на нього мишею (дуже модний нині прийом), то для такого елемента повинні бути вказані два атрибути:

`onmouseover = "Программа1 (" параметр1 ")"`
`onmouseout = " Программа2 ( "параметр2") "`

Перша програма (сценарій) змінить вигляд елемента, коли курсор миші опиниться над ним, а друга поверне елементу колишній вигляд, коли курсор миші буде прибраний. Різні елементи дозволяють використовувати різні події.

Події, пов'язані з мишею:

- `onclick` - клацання мишею на елементі;
- `ondblclick` - подвійне клацання мишею на елементі;
- `onmousedown` - кнопка миші натиснута;
- `onmouseup` - кнопка миші відпущена;
- `onmousemove` - покажчик миші переміщений в область елемента;
- `onmouseover` - покажчик миші розташований над елементом;
- `onmouseout` - покажчик миші переміщений за межі області елемента.

Події, пов'язані з вибором елементів і редагуванням форм:

- `onfocus` - елемент обраний (отримано фокус);
- `onselect` - частина тексту всередині елемента виділена;
- `onchange` - дані в елементі були змінені;

- `onblur` - елемент перестав бути обраним (втрачений фокус).

Події, пов'язані з клавіатурою:

- `onkeydown` - клавіша натиснута;
- `onkeyup` - клавіша відпущена;
- `onkeypress` - клавіша натиснута і відпущена.

6. Основні елементи для форматування тексту

Текст - єдиний об'єкт Web-сторінки, який не вимагає спеціального визначення. Іншими словами, довільні символи інтерпретуються за замовчуванням як текстові дані. Але для форматування тексту існує велика кількість елементів. Більшість з них, крім спеціальних, підтримує стандартні атрибути: `id`, `class`, `lang`, `dir`, `title`, `style` і атрибути подій.

Спочатку в `html` було введено менше можливостей для форматування тексту, ніж в звичайні текстові редактори. В результаті авторам гіпертекстових документів доводилося вдаватися до різних хитрощів, щоб надати тексту заданий вид. Зараз ситуація змінилася, але всі додаткові можливості здійснюються за рахунок застосування таблиць стилів. Наприклад, тільки за допомогою властивості `text-indent` можна задати величину відступу першого рядка абзацу.

Форматувати текст можна і за допомогою традиційних елементів: виділяти фрагменти курсивом, напівжирним, вибирати шрифт і т. Д. Розглянемо ці елементи. Для них можуть бути використані стандартні атрибути `id`, `class`, `Lang`, `dir`, `title`, `style`, атрибути подій, а також атрибути, що визначають унікальні властивості певних елементів.

`<p> </p>`

Елемент абзацу (`paragraph`) - один з найкорисніших. Він дозволяє використовувати тільки початковий тег, так як наступний елемент `P` позначає не тільки початок наступного абзацу, а й кінець попереднього. У тих випадках, коли за змістом необхідно позначити завершення абзацу, можна використовувати і кінцевий тег. У деяких випадках початковий тег зручно ставити в кінці рядка: він не тільки позначить кінець абзацу, а й виконає функцію тега `<BR>` (розрив рядка). наприклад:

`<p>` Текст першого абзацу. `<p>` Текст другого абзацу. `</p>` Текст третього абзацу. `<p>`

Разом з елементом абзацу можна використовувати атрибут вирівнювання `align`:

- `align = "left"` - вирівнювання по лівому краю;
- `align = "center"` - вирівнювання по центру;
- `align = "right"` - вирівнювання по правому краю.

Для центрування абзацу слід використовувати таку конструкцію:

`<p align = "center">` Текст абзацу

Абзаци форматуються оглядачем, і їх вид залежить, зокрема, від розміру вікна програми. Три наступних елемента дозволяють внести деяку визначеність у формат абзацу.

`<br>`

Елемент, що забезпечує примусовий перехід на новий рядок. Він має тільки початковий тег. У місці його розміщення рядок закінчується, а залишився текст друкується з нового рядка.

Атрибут `clear` дозволяє вирівнювати об'єкти (наприклад, малюнки) щодо тексту, в якому використаний елемент `BR`. Якщо елемент об'єкта містить атрибут `align`, то в розташованих поруч елементах `BR` повинен бути присутнім атрибут `clear`, наприклад:

`<BR clear="Right">`

Значення атрибута:

- `none` - значення за замовчуванням;
- `left` - якщо об'єкт вирівняний по лівому краю;
- `right` - якщо об'єкт вирівняний вправо;
- `all` - для об'єкта, який може бути вирівняний по будь-якому краю.

Стандартні атрибути: `id`, `class`, `title`, `style`.

<nobr> </nobr>

Цей елемент по своїй дії є прямою протилежністю попереднього. Текст, укладений між його тегами, буде виведений в один рядок. Якщо довга рядок не вміститься на екрані, для її перегляду доведеться використовувати горизонтальну смугу прокрутки.

<pre> </pre>

Елемент для позначення тексту, відформатованого заздалегідь (preformatted). Мається на увазі, що текст буде виведений в тому вигляді, в якому був підготовлений автором. Наприклад, враховуються символи кінця рядка, що з'явилися при наборі тексту в редакторі. У всіх інших випадках браузер ігнорує ці символи. Можливий і зворотний ефект: якщо користувач введе текст як одну довгу рядок, то вона не буде розірвана оглядачем, а піде за край вікна програми. У цьому сенсі елемент PRE працює так само, як елемент NOBR. За замовчуванням для відформатованого заздалегідь тексту вибирається моношірний шрифт. Цей елемент зручно використовувати для показу лістингів програм або для виведення текстових документів, переформатування яких може призвести до спотворення їх сенсу.

Елемент PRE дозволяє набрати текст з використанням спеціальних символів форматування, таких як «line feed» або «carriage return» (див. Табл. 3.1 нижче).

Теоретично можна уявити ситуацію, коли розробнику Web-сторінки потрібно показати, як створювали лінії таблиць в далекому минулому, коли текстовий режим вже існував, а символи псевдографіки ще не були винайдені. У хід йшли плюси, знаки оклику і тире. У цьому випадку елемент PRE також виявиться незамінний, хоча я не рекомендую піддаватися ностальгічним поривам: краще зробити чорно-білий малюнок формату GIF.

Для цього елемента визначений спеціальний атрибут, який дозволяє задати ширину блоку тексту в символах:

width = число символів

Цей атрибут не підтримує багатьма браузерами. Стандартні атрибути: id, class, lang, dir, title, style, атрибути подій.

<center> </center>

Елемент для центрування тексту, а точніше - будь-якого вмісту. Цей елемент не є загальнозживаним. У тих випадках, коли це можливо, замість нього в елементах тексту використовують атрибут align = "center".

** **

Жирний шрифт. Дуже популярний елемент. Використання напівжирного шрифту - прийом, запозичений з текстових редакторів.

<big> </big>

Збільшення розміру шрифту.

<small> </small>

Зменшення розміру шрифту.

<i> </i>

Виділення тексту курсивом.

<strike> </strike> або <S> </s>

Закреслена накреслення тексту. В даний час елемент STRIKE замінюють простішим в написанні елементом S.

<u> </u>

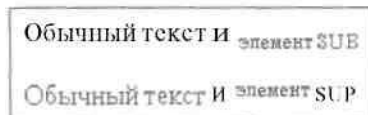
Підкреслена накреслення тексту.

Елемент, що створює ефект нижнього індексу (subscript).

Елемент, що створює ефект верхнього індексу (superscript).

Дія двох останніх елементів ілюструє фрагмент файлу гіпертексту Text.htm, показаний на рис. 3.1. Обидва ці елементи забезпечують зменшення розміру шрифту. Тому

їх можна використовувати і для форматування абзацу цілком, якщо треба, щоб він був виведений дрібним шрифтом.



Мал. 1.2. Використання елементів SUB і SUP

**<ins> </ins> і **

Ці елементи дозволяють виділити текст, який треба позначити як вставлений (елемент INS) або віддалений (елемент DEL). Візуально вставлений текст виділяється підкресленням, а віддалений - зачеркиванням.

Для вказівки джерела змін, тобто для документа, в якому знаходиться даний фрагмент або дано пояснення, чому в тексті з'явилася така вставка, може бути використаний атрибут:

cite = "Адреса (URL)"

Для дати зміни теж передбачений спеціальний атрибут:

datetime = "дата"

В результаті початковий тег може мати такий вигляд:

<INS datetime = "2000" 04-26 "cite = "[file: /// C: / Pages / Додатки. htm](file:///C:/Pages/Додатки.htm) ">

<BASEFONT>

Елемент, що визначає базовий (основний для всієї сторінки) розмір шрифту. У середині елемента необхідно вказати атрибут:

size = базовий розмір шрифту

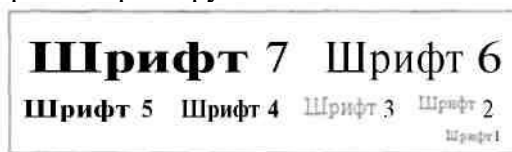
Величина для цього атрибута може лежати в межах від 1 до 7. За замовчуванням використовується величина 3. Установка, виконувана цим елементом, має значення для елемента FONT (див. Нижче), який дозволяє задавати відносний розмір шрифту. Інші атрибути у цього елемента такі ж, як і у елемента FONT.

**< font > **

Визначення типу, розміру і кольору шрифту. Всі ці характеристики визначаються за допомогою відповідних атрибутів. Абсолютний розмір шрифту задається атрибутом *size* (розмір):

Size = абсолютний розмір шрифту

Цей атрибут може приймати значення від 1 до 7. На рис. 3.2 показані кілька зразків написів, виконаних шрифтами різного розміру.



Мал. 1.3. Можливі варіанти розмірів шрифту

Розмір шрифту може також задаватися щодо базового:

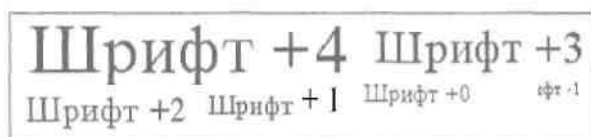
size = + число

size = число

При призначенні величини для цього атрибута треба враховувати величину базового розміру. У сумі ці дві величини повинні відповідати одному з абсолютних розмірів. Так, для базового розміру, рівного 3, відносний розмір може перебувати в межах від -2 до +4. Якщо величина виходить за допустиму межу, то використовується або шрифт розміру 7, або шрифт розміру 1. На рис. 3.3 показані написи, виконані шрифтами із заданим відносним розміром.

Для елемента FONT можна використовувати атрибут кольору:

color = "колір"



Мал. 1.4. За допомогою відносних величин теж можна отримати сім градацій розміру шрифту

Атрибут `face` (вид) дозволяє задавати певний шрифт або кілька шрифтів (через крапку з комою), наприклад:

`face = "Arial; Verdana; Tahoma"`

Правда, є одна проблема. Web-сторінки переглядає безліч людей і немає гарантії, що у кожного з них виявиться потрібний шрифт. Якщо в системі не встановлений шрифт точно з такою ж назвою, то браузер використовує стандартний шрифт з числа призначених за замовчуванням: один пропорційний, інший моношірний.

Елемент `FONT` може з успіхом замінити елементи заголовка `H1 ... H6`. Для останніх, наприклад, не передбачено завдання кольору букв. Щоб заголовок, створений на основі елемента `FONT`, добре виглядав, цей елемент необхідно комбінувати з іншими: `CENTER`, `B`, `I`, `P` і т. Д. Додаткові атрибути: `id`, `class`, `lang`, `dir`, `title`, `style`.

7. Табуляція, прогалини, переноси .

У табл. 1.1 наведені коди деяких, здебільшого невидимих в звичайному режимі перегляду, символів. Ряд символів (таких як «line feed» або «carriage return») використовується в текстових редакторах для форматування тексту, а користувач їх не бачить. Такі символи можна вказувати всередині елемента `PRE`, де вони будуть виконувати свою функцію.

У тексті можуть бути використані символи м'якого переносу. Вони показують, де може бути закінчена рядок, але такі символи не враховуються при виконанні операцій пошуку і сортування. Якщо в місці розташування м'якого переносу рядок не закінчується, символ перенесення видно не буде. Це відрізняє варіант переносу від звичайного дефіса, який видно завжди.

Цікавий і символ пропуску нульової довжини. Такі символи використовуються для прихованого поділу слів.

Таблиця 1.1. коди символів

код символу	Числовий код (html)	Назва
9			табулятор
10	
	Кінець рядка (line feed)
12		Кінець сторінки (form feed)
13		Повернення каретки (carriage)
32	 	пропуск
45	-	дефіс
160	 	нерозривний пробіл
173	­	м'який перенос
8203	​	Пропуск нульовий ширини

8. Гіперпосилання <a>

Один з найважливіших елементів мови, що забезпечує створення гіперпосилань. Найчастіше використовується такий шаблон:

довільний текст `<A href = 'Адреса посилання'>` текст для клацання `</a>`

Або такий:

`<A href = "Адреса посилання"> <IMG src = "Посилання на малюнок"> </a>`

Перший шаблон використовується в тому випадку, коли гіперпосилання зустрічається в тексті. Атрибут `Href` може вказувати на ресурс Інтернету, файл на

локальному диску або на мітку всередині поточної сторінки. Текст, розташований всередині елемента А, являє собою видиму частину гіперпосилання. На ньому повинен клацнути користувач, щоб здійснити перехід. Броузер виділяє цей фрагмент кольором, а після використання гіперпосилання змінює колір, щоб забезпечити підказку.

Другий шаблон задається в тому випадку, коли видима частина гіперпосилання являє собою малюнок. Якщо для останнього визначена рамка, то вона теж змінює колір після використання гіперпосилання. Якщо посилання вказує на малюнок, який знаходиться на локальному диску, вона обов'язково повинна починатися зі слова file, тобто містити вказівку на протокол:

href = "[file://Диск:\Шлях до файлу](#)"

або

href = "[file:///Диск:/Шлях до файлу](#)"

За замовчуванням використовується посилання на файли поточної папки (тієї, де розташований файл Web-сторінки). У цьому випадку просто вказується ім'я файлу, наприклад: page2.html, strelka.gif, photo35.jpg.

Часто використовуються відносні посилання на папки: це дозволяє легко міняти місце розташування комплексу сторінок на диску. Якщо в цій папці є інша, в якій розміщені необхідні файли, посилання будується за таким шаблоном:

Href = "[./ Папка / Файл.min](#)"

Тут на структуру вкладених папок вказує точка перед похилою рисою. Якщо необхідно вказати папку, яка знаходиться на тому ж рівні вкладки, що і поточна, то додають ще одну точку:

Href = "[../ Папка / Файл.min](#)"

Подібно до багатьох інших елементів мови, елемент А вимагає використання атрибутів. Атрибут гіперпосилання ми вже знаємо, шаблон його такий:

href = "URL" або

href = "Протокол: // Адреса посилання"

наприклад:

href = "[http://www.netscape.com](#)"

Кодове слово, що стоїть на початку URL, позначає так звану схему доступу. Вона визначає тип сервера, доступний за допомогою даної посилання. Для користувача це представляється як доступ до однієї з різновидів Інтернету. У цьому -смысле можна сказати, що Інтернет - це як би кілька мереж в одній. У кожній з цих приватних мереж існують свої правила доступу, переваги, недоліки, свої прихильники і противники. Але всі користувачі використовують одні і ті ж канали зв'язку. Схожа ситуація спостерігається і в звичайних телефонних мережах. Вони можуть служити для зв'язку голосом, передачі факсів, міжкомп'ютерної зв'язку і т. Д.

WWW як найсучасніша система, повинна забезпечувати сумісність з більш старими системами, тому від старих протоколів не відмовляються, а намагаються пристосувати їх до сучасних потреб (наприклад ftp). Існують наступні схеми доступу:

- file - доступ до файлу на локальному диску;
- ftps - доступ до архівів файлів по протоколу передачі файлів (file transfer protocol);
- https - доступ до WWW;
- mailto -відправка повідомлення по електронній пошті;
- news - доступ до новин USENET;
- nntp - доступ до новин USENET по протоколу NNTP;
- telnet - підключення по протоколу telnet;
- wais - підключення до системи пошуку WAIS.

Коли гіперпосилання використовується для вказівки адреси електронної пошти, її вибір забезпечує не перехід до нового документу, а запуск діалогу для відправки повідомлення зазначеному адресату. Зазвичай таке посилання розміщують в кінці сторінки

для забезпечення зв'язку з Web-майстром або автором сторінки. Для своєї сторінки я б міг скласти таку посилання:

`<A href = "mailto:goncharov@online.ru">Олексій Гончаров </A>`

У тому випадку, коли використовуються переходи всередині поточної сторінки, на ній повинні бути розставлені мітки:

`<A name = "Мітка"> </a>`

У великих сайтах часто використовуються мітки для переходу до певної частини деякої сторінки:

`<A name = "Http://Адрес/Файл.html#метка"> </a>`

Для переходу до мітки використовується посилання за таким шаблоном (приклад наведено в розділі «Анатомія Web-сторінки»:

Текст підказки `<A href = "# Метка"> Текст для клацання </a>`

Для елемента А передбачені різні атрибути. Атрибут hreflang, за аналогією з атрибутом lang, дозволяє вказати мову, який використовується на адресується сторінці.

У структуру гіперпосилань закладена можливість створення складних текстових документів, доступних через Інтернет. Передбачається, що такі документи будуть складатися з багатьох html-сторінок з перехресними посиланнями. Щоб користувач міг ефективно управляти документом, броузер повинен оптимізувати роботу з окремими сторінками, наприклад, завантажувати сторінки, яка може знадобитися, в фоновому режимі. Для цього необхідно забезпечити сторінки інформацією про призначення посилань.

Для вирішення цього завдання гіперпосилання поділяються на прямі (forward) і зворотні (reverse). Посилання, що викликає перехід з поточної сторінки, називається прямою. Відповідно, за допомогою броузера або іншого посилання може бути виконаний і зворотний перехід. Для визначення більш точного типу посилання використовуються два атрибути (один для прямих, інший - для зворотних посилань).

`rel = "Ілпрямой посилання" rev = "Тві зворотного посилання "`

Визначено такі стандартні типи посилань:

- alternate - інша версія документа;
- stylesheet - таблиця стилів у вигляді окремого файлу;
- start - перша сторінка в структурі документа;
- next - наступна (в сенсі виконання переходів) сторінка;
- prev - попередня (в сенсі виконання переходів) сторінка;
- contents - сторінка, на якій знаходиться зміст всього документа;
- index - сторінка, на якій знаходиться алфавітний покажчик;
- glossary - сторінка, на якій знаходиться словник термінів;
- copyright - інформація про авторські права на документ;
- chapter - ознака глави документа;
- section - ознака розділу документа;
- subsection - ознака підрозділу документа;
- appendix - ознака додатки документа;
- help - довідкові дані документа;
- bookmark - закладка всередині документа.

Існують атрибути, які характерні тільки для певних конструкцій. Атрибути Shape і coords використовуються в картах (див. Розділ «Малюнки та карти» глави 4). Атрибут target буває дуже корисним при створенні фреймів (див. Розділ «Фрейми» поточної глави). Атрибути accesskey і tabindex можна вказувати, якщо елемент А входить до складу форм (див. Розділ «Елементи форм» глави 4).

Елемент А дозволяє використовувати і стандартні атрибути: id, class, lang, dir, title, type, style, атрибути подій.

<link>

На відміну від атрибута А, який вказується в тексті сторінки, елемент link використовується в заголовку сторінки, тобто всередині елемента head.


```
<head>
<title> Глава 1 </title>
<link rel = "prev" href = "vvdenie.htm ">
<link rel = "next" href = "page1.htm">
<link rel = "index" href = "ukazatel.htm-->
</head>
```

Елемент link не створює гіперпосилань в тексті сторінки, тому для визначення об'єкта, на якому можна клацнути мишею, необхідно використовувати елемент A з атрибутом href, який має те ж призначення, що і в елементі link.

Атрибути використовуються, в основному, такі ж, як і в елементі A: charset, href, hreflang, id, class, lang, dir, media, rel, rev, style, target, title, type, атрибути подій.

Контрольні питання.

1. **Структура документа:** Опишіть обов'язкову структуру HTML-файлу. Які чотири основні теги (<html>, <head>, <body>, <!DOCTYPE>) формують його каркас і за що відповідає кожен з них?
2. **Мета-інформація:** Які атрибути приймає тег <meta>? Поясніть призначення атрибутів name="Keywords", name="Description" та http-equiv="Content-Type".
3. **Атрибути тіла документа:** Які параметри можна задати в тегу <body> для глобального налаштування кольорів (фону, тексту, звичайних посилань, активних та відвіданих посилань)?
4. **Заголовки:** Скільки рівнів заголовків підтримує HTML (<h1>...<h6>) і як за допомогою атрибута align змінити їх вирівнювання на сторінці?
5. **Форматування тексту:** У чому полягає різниця між тегами фізичного форматування: , <i>, <u>, <strike>, <sub>, <sup>? Наведіть приклади використання верхнього та нижнього індексів.
6. **Шрифти:** Як працює тег ? Опишіть дію його атрибутів size (діапазон значень від 1 до 7), color та face.
7. **Абзаци та переноси:** Чим відрізняється використання тегу <p> від тегу
? Які параметри вирівнювання (left, center, right, justify) доступні для абзацу?
8. **Преформатований текст:** Для чого використовується тег <pre> і як він обробляє пробіли та переноси рядків на відміну від звичайного тексту?
9. **Гіперпосилання (Якорі):** Як створити посилання на зовнішній ресурс за допомогою тегу <a>? Поясніть різницю між атрибутами href (адреса) та name (якір всередині сторінки).
10. **Типи посилань:** Які протоколи можна використовувати в атрибуті href (наприклад, http://, ftp://, mailto:, file://) і що відбудеться при їх використанні?
11. **Спеціальні символи:** Як в HTML відобразити зарезервовані символи, такі як "менше" (<), "більше" (>) та нерозривний пробіл, використовуючи спецкоди?
12. **Коментарі:** Який синтаксис використовується для додавання коментарів у код HTML, щоб вони не відображалися у браузері?